`imass-worktree`

1. ¿Qué es?

`imass-worktree` es una herramienta y estándar operativo desarrollado en **Python >= 3.11** para gestionar workspaces de desarrollo aislados mediante **Git worktrees**.

Su modelo principal establece una relación determinista:

```
target → branch → worktree
```

El ejecutable de consola es:

```
imass-wt
```

Su objetivo es permitir trabajar en varias tareas o ramas simultáneamente mediante **ecosistemas de desarrollo temporales e independientes por target**, especialmente útiles cuando varios desarrolladores comparten una misma infraestructura, como una EC2.

Cada workspace puede aislar no sólo el código y la rama, sino también su **runtime, puertos, contenedores, dependencias, base de datos y exposición externa**, evitando interferencias entre desarrolladores. Esto permite realizar con seguridad operaciones potencialmente destructivas —por ejemplo `migrate:fresh`, pruebas que modifican datos o reconstrucciones del entorno— sin afectar el ambiente principal de desarrollo ni los workspaces de otros desarrolladores.

El entorno puede crearse, utilizarse y posteriormente destruirse de forma controlada.

---

2. Arquitectura general

La arquitectura sigue esta separación de responsabilidades:

```
core genérico
+ configuración del proyecto cliente
+ capabilities opcionales
+ integrations opcionales
```

Esto permite que `imass-worktree` sea independiente del stack tecnológico del proyecto.

El **core no depende de Laravel, PHP, Node, Docker, tmux, Kiro ni ninguna otra tecnología específica**.

3. Estructura principal del código

Dentro de:

```
src/imass_worktree/
```

se encuentran los siguientes componentes.

`cli.py`

Superficie pública del CLI.

Responsabilidades principales:

- parser de comandos;
- dispatch;
- render de resultados;
- único lugar encargado de escribir en `stdout` y `stderr`.

Los handlers son delgados y devuelven objetos `CommandResult`.

`core/`

Contiene el lifecycle principal:

```
init
adopt
destroy
list_workspaces
status
startstop
doctor
```

Además contiene componentes como:

```
errors.py
result.py
locking.py
naming.py
project.py
```

```
target.py
worktree.py
```

config/

Gestiona la configuración del proyecto:

```
loader
models
validation
target
defaults
```

capabilities/

Implementa comportamientos opcionales:

```
processes  (procesos administrados del workspace)
tmux       (sesión persistente por target)
runtime    (configura el entorno del target: .env, identidad Docker Compose y contenedores/servici
ports      (puertos aislados por workspace)
tunnels    (exposición externa, por ejemplo mediante zrok)
data       (base de datos aislada por target)
bootstrap  (dependencias y preparación inicial del workspace)
```

Incluye además:

```
registry.py
base.py
```

integrations/

Integraciones con herramientas externas:

```
kiro
vscode
codex
claude
```

Incluye adapters, registry y service.

git/

Boundary encargado de interactuar con Git:

```
lifecycle
models
repository
runner
```

`security/`

Controles de seguridad para filesystem y procesos:

```
paths.py
process.py
```

Entre sus componentes se encuentran:

```
SafePathResolver
SafeProcessRunner
```

Los boundaries de rutas son conscientes de symlinks.

`transactions/`

Gestiona operaciones transaccionales y rollback:

```
transaction.py
LifecycleTransaction
```

`hooks/`

Gestiona los eventos del lifecycle:

```
events.py
runner.py
```

Por ejemplo:

```
after_init
after_adopt
```

---

4. Comandos disponibles

La superficie pública del CLI está definida por una familia de comandos estable.

Comandos que requieren `TARGET`

Comando	Función
`imass-wt init <target>`	Crea un workspace nuevo: branch + worktree + recursos opcionales.
`imass-wt adopt <target>`	Incorpora trabajo existente como workspace administrado.
`imass-wt status <target>`	Muestra el estado del workspace.
`imass-wt start <target>`	Reactiva los recursos asociados al workspace.
`imass-wt stop <target>`	Detiene recursos sin destruir el workspace ni el trabajo.
`imass-wt destroy <target>`	Retira de forma segura el workspace y sus recursos administrados.

Comandos que no requieren `TARGET`

Comando	Función
`imass-wt list`	Lista los workspaces administrados.
`imass-wt doctor`	Diagnostica si el proyecto y el entorno están correctamente configurados.

5. Ejemplo de uso

Un lifecycle normal puede verse así:

```
# Validar configuración y entorno
imass-wt doctor
```

```
# Crear un workspace
imass-wt init TASK-123

# Consultar los workspaces existentes
imass-wt list

# Consultar un workspace concreto
imass-wt status TASK-123

# Detener temporalmente sus recursos
imass-wt stop TASK-123

# Reactivarlos
imass-wt start TASK-123

# Eliminar el workspace cuando terminó el trabajo
imass-wt destroy TASK-123
```

Conceptualmente:

```
doctor
   ↓
init TASK-123
   ↓
trabajar
   ↓
status / list
   ↓
stop
   ↓
start
   ↓
destroy
```

---

6. `init`

```
imass-wt init <target>
```

Crea un workspace nuevo.

Dependiendo de la configuración del proyecto puede encargarse de:

```
crear branch
    ↓
crear worktree
    ↓
reservar puerto
    ↓
preparar runtime
    ↓
preparar base de datos
    ↓
instalar dependencias
    ↓
crear sesión tmux
    ↓
abrir integración
```

En este contexto, **preparar runtime** significa configurar el entorno de ejecución propio del target:

```
crear/configurar el runtime del target
→ materializar su .env
→ asignarle su identidad de Docker Compose
→ preparar sus contenedores/servicios
```

No todas estas operaciones son obligatorias.

Las adicionales dependen de las capabilities habilitadas en:

```
.imass-worktree.toml
```

---

7. `adopt`

```
imass-wt adopt <target>
```

Permite incorporar trabajo existente al modelo administrado por `imass-worktree`.

Por ejemplo, si ya existe una rama compatible:

```
branch existente
      ↓
imass-wt adopt
      ↓
worktree administrado
```

```
      ↓
provisioning configurado
```

Es útil para migrar trabajo existente sin tener que comenzar nuevamente con `init`.

---

8. `status`

```
imass-wt status <target>
```

Permite inspeccionar el estado de un workspace y de las capabilities asociadas.

Puede informar, entre otras cosas:

```
branch
worktree
runtime
puerto
tmux
procesos
túnel
data
bootstrap
```

dependiendo de la configuración del proyecto.

---

9. `list`

```
imass-wt list
```

Lista los workspaces administrados actualmente.

Es especialmente útil cuando existen varias tareas simultáneas:

```
TASK-101
TASK-102
TASK-103
```

Cada una puede tener su propia:

```
branch
worktree
runtime
puerto
base de datos
sesión tmux
procesos
```

---

10. `stop`

```
imass-wt stop <target>
```

Detiene los recursos operativos asociados al workspace sin destruirlo.

Por ejemplo, dependiendo de las capabilities:

```
detener procesos
cerrar túnel
detener runtime
gestionar sesión tmux
```

La operación preserva:

```
branch
worktree
commits
cambios locales
```

Por tanto:

```
stop ≠ destroy
```

`stop` significa **pausar el entorno**, no eliminar el trabajo.

---

11. `start`

```
imass-wt start <target>
```

Reactiva los recursos de un workspace previamente detenido.

Por ejemplo:

```
runtime
procesos
tmux
túnel
```

dependiendo de las capabilities habilitadas.

---

12. `destroy`

```
imass-wt destroy <target>
```

Retira de forma segura un workspace.

Su comportamiento es deliberadamente conservador.

Si existe trabajo local que pudiera perderse, la operación debe bloquearse en lugar de destruirlo silenciosamente.

Conceptualmente:

```
verificar seguridad
       ↓
detener/limpiar recursos
       ↓
liberar puerto
       ↓
retirar worktree
       ↓
gestionar branch según configuración
```

---

13. `doctor`

```
imass-wt doctor
```

Diagnostica si el proyecto y el entorno están preparados para trabajar con `imass-worktree`.

Puede validar:

```
configuración
Git
branch base
política de targets
paths
capabilities
dependencias externas
runtime
tmux
Composer
Docker
etc.
```

La meta antes de empezar a trabajar normalmente es obtener:

```
Global: PASS
```

o únicamente advertencias conocidas y justificadas.

---

14. Integración con herramientas externas

`init` y `start` pueden utilizar una integración mediante:

```
--open <herramienta>
```

Por ejemplo:

```
imass-wt init TASK-123 --open kiro
```

o:

```
imass-wt start TASK-123 --open vscode
```

Las integraciones disponibles incluyen:

```
kiro
vscode
codex
claude
```

Estas herramientas se abren tomando como contexto el worktree correspondiente.

---

15. Capabilities

Las capabilities son funcionalidades opcionales.

Sólo participan si el proyecto las habilita en:

```
.imass-worktree.toml
```

Esto significa que un proyecto simple puede utilizar únicamente Git worktrees, mientras que un proyecto más complejo puede automatizar prácticamente todo su entorno de desarrollo.

Las principales son:

```
processes  (procesos administrados del workspace)
tmux       (sesión persistente por target)
runtime    (configura .env, identidad Docker Compose y contenedores/servicios del target)
ports      (puertos aislados para evitar colisiones)
tunnel     (exposición externa, por ejemplo mediante zrok)
data       (base de datos independiente por target)
bootstrap  (instalación de dependencias y validación de readiness)
```

15.1. `runtime`

Permite administrar un runtime por workspace.

En un entorno basado en Docker Compose, su responsabilidad incluye:

```
crear/configurar el runtime del target
→ materializar su .env
→ asignarle su identidad de Docker Compose
→ preparar sus contenedores/servicios
```

Esto permite que cada target mantenga su propio entorno de ejecución sin confundirse con el runtime principal ni con el de otros workspaces.

Drivers disponibles:

```
none
docker-compose
```

Con Docker Compose puede encargarse de:

```
provision
start
stop
cleanup
```

Esto permite que cada target tenga su propio runtime aislado.

15.2. `ports`

Reserva puertos por target.

Esto permite ejecutar varios workspaces simultáneamente sin colisiones.

Ejemplo conceptual:

```
TASK-101 → 8100
TASK-102 → 8101
TASK-103 → 8102
```

15.3. `tmux`

Administra una sesión `tmux` persistente por target.

Ejemplo conceptual:

```
TASK-101 → sesión tmux TASK-101
TASK-102 → sesión tmux TASK-102
```

Esto permite mantener procesos o herramientas funcionando aunque se cierre la terminal o se pierda una conexión SSH.

15.4. `data`

Permite provisionar datos aislados por target.

Actualmente dispone de provider:

```
postgres
```

Puede tomar la base del runtime principal y preparar una base aislada para el workspace.

Conceptualmente:

```
PostgreSQL base
      ↓
data provisioning
      ↓
PostgreSQL TASK-123
```

Esto permite que el workspace realice pruebas, migraciones u operaciones destructivas sobre su propia copia sin afectar la base principal de desarrollo ni a otros desarrolladores.

15.5. `bootstrap`

Prepara dependencias y valida que el workspace esté listo para trabajar.

Actualmente dispone de provider:

```
composer
```

Puede ejecutar automáticamente la instalación de dependencias durante:

```
init
adopt
```

Por ejemplo:

```
composer install
```

Posteriormente puede verificar artifacts requeridos, como:

```
vendor/autoload.php
vendor/bin/pest
vendor/bin/pint
vendor/bin/deptrac
```

según la configuración del proyecto.

15.6. `tunnel`

Permite exponer temporalmente un endpoint local mediante un túnel.

Provider disponible:

```
zrok
```

Conceptualmente:

```
workspace
   ↓
puerto local
   ↓
zrok
   ↓
URL pública
```

Esto permite publicar cada workspace de forma independiente sin hacer que diferentes desarrolladores o targets compitan por el mismo endpoint.

15.7. `processes`

Permite gestionar procesos asociados a cada workspace.

Por ejemplo:

```
workers
watchers
development servers
background processes
```

Pueden iniciarse y detenerse automáticamente según el lifecycle configurado.

---

16. Cómo adoptar `imass-worktree` en un proyecto

La adopción consiste principalmente en:

```
instalación
    +
configuración
    +
validación
```

No se debe copiar el core dentro del proyecto cliente ni modificarlo específicamente para ese proyecto.

Paso 1. Instalar `imass-worktree`

Instalar la herramienta y verificar que el CLI esté disponible.

Por ejemplo:

```
pipx install imass-worktree
```

Verificar:

```
imass-wt --version
```

Paso 2. Crear la configuración del proyecto

Crear en la raíz del repositorio:

```
.imass-worktree.toml
```

Paso 3. Configurar las reglas básicas

Como mínimo deben definirse aspectos como:

```
project.base_branch
target.regex
target.example
branch.pattern
worktree.directory
worktree.pattern
```

Ejemplo conceptual:

```
[project]
name = "mi-proyecto"
base_branch = "main"

[target]
regex = "^[a-z0-9-]+$"
example = "task-123"

[branch]
pattern = "feature/{target}"

[worktree]
directory = ".local-wt"
pattern = "{target}"
```

Paso 4. Habilitar capabilities necesarias

Sólo deben habilitarse las que el proyecto realmente necesite.

Por ejemplo:

```
runtime    (configura .env, identidad Docker Compose y contenedores/servicios)
ports      (puertos aislados)
tmux       (sesión persistente)
data       (base de datos independiente)
```

```
bootstrap  (dependencias y readiness)
tunnel     (exposición externa con zrok)
processes  (procesos administrados)
```

También pueden habilitarse integrations:

```
kiro
vscode
codex
claude
```

Un proyecto Git-only puede no requerir ninguna capability.

Paso 5. Validar el proyecto

Ejecutar:

```
imass-wt doctor
```

La configuración y los prerrequisitos deben quedar correctamente validados antes de utilizar normalmente el lifecycle.

Paso 6. Probar el lifecycle

Ejecutar un ciclo básico:

```
imass-wt init mi-target
imass-wt status mi-target
imass-wt list
imass-wt stop mi-target
imass-wt start mi-target
imass-wt destroy mi-target
```

Paso 7. Versionar la configuración

Agregar al repositorio:

```
.imass-worktree.toml
```

La configuración compartida **no debe contener secretos**.

Después debe documentarse para el equipo la forma estándar de utilizar:

```
init
adopt
status
list
start
stop
destroy
doctor
```

---

17. Perfiles posibles de adopción

Proyecto Git-only

Sólo necesita:

```
Git
branch
worktree
```

El lifecycle puede ser:

```
imass-wt init
imass-wt status
imass-wt list
imass-wt destroy
```

No requiere Docker, tmux ni ninguna otra herramienta opcional.

Aplicación con Docker Compose

Puede habilitar:

```
ports      (puertos independientes)
runtime    (entorno Docker Compose independiente)
tmux       (sesión persistente)
data       (copia aislada de PostgreSQL)
bootstrap  (dependencias del workspace)
tunnel     (exposición externa independiente)
```

Un `init` puede entonces convertirse en:

```
crear branch
     ↓
crear worktree
     ↓
reservar puerto
     ↓
crear/configurar runtime Docker
     ↓
materializar .env
     ↓
asignar identidad Docker Compose
     ↓
preparar PostgreSQL independiente
     ↓
ejecutar Composer
     ↓
crear sesión tmux
     ↓
workspace listo
```

CLI o librería

Puede utilizar solamente:

```
Git
worktrees
```

y añadir procesos o bootstrap únicamente cuando sean necesarios.

Docker no es un requisito de `imass-worktree` .

---

18. Contrato de códigos de salida

Los códigos de salida forman parte del contrato estable de la herramienta.

Código	Identificador	Significado
0	OK	Operación exitosa.
2	USAGE_ERROR	Error de sintaxis o argumentos del CLI.
3	CONFIGURATION_ERROR	Configuración ausente o inválida.
4	STATE_ERROR	El estado del workspace no permite realizar la operación.
5	SAFETY_BLOCKED	Una regla de seguridad impide continuar.
6	OPERATION_FAILED	Falló una operación o dependencia esperada.
70	INTERNAL_ERROR	Error interno inesperado.
78	NOT_IMPLEMENTED	Reservado para superficies futuras.

19. Modelo de errores

Los errores esperados están modelados mediante:

`core/errors.py`

con:

`ImassWorktreeError`

y sus subclases.

Cada error dispone de:

```
code
exit_code
```

Los errores asociados a mutaciones parciales permiten que el sistema conozca cuándo una operación pudo haber modificado recursos antes de fallar.

Esto se integra con el modelo transaccional y de rollback.

---

20. Seguridad

`imass-worktree` define un conjunto de invariantes de seguridad que el lifecycle debe respetar.

`INV-001` — No pérdida silenciosa de trabajo

Nunca debe eliminarse trabajo local silenciosamente.

`INV-002` — `stop` preserva el workspace

`stop` conserva:

```
worktree
branch
cambios locales
```

`INV-003` — `destroy` es conservador

Ante una situación incierta, debe bloquear antes que destruir.

`INV-004` — `init` no sobrescribe recursos existentes

No debe sobrescribir:

```
branches
worktrees
directorios
```

existentes.

`INV-005` — Un fallo nunca es éxito

Una operación que falla no debe reportarse como exitosa.

`INV-006` — Fallos parciales diagnosticables

Si una operación modifica parcialmente el entorno antes de fallar, el estado debe poder diagnosticarse.

`INV-007` — Core agnóstico del stack

El core no debe depender de:

```
PHP
Laravel
Node
Docker
tmux
Kiro
etc.
```

`INV-008` — Secretos fuera de configuración versionable

Los secretos no deben almacenarse en:

```
.imass-worktree.toml
```

`INV-009` — Capabilities realmente opcionales

Una capability no habilitada no debe convertirse en requisito indirecto.

`INV-010` — Autoridad externa antes que estado duplicado

Cuando Git o un recurso real puede determinar el estado, se prefiere consultar esa fuente antes que mantener una segunda fuente de verdad.

---

21. Concurrencia y aislamiento

La herramienta está diseñada para soportar múltiples workspaces simultáneos dentro de una misma infraestructura compartida.

Por ejemplo:

```
TASK-A
├── branch A
├── worktree A
├── puerto A
├── runtime A
├── base de datos A
└── endpoint/túnel A

TASK-B
├── branch B
├── worktree B
├── puerto B
├── runtime B
├── base de datos B
└── endpoint/túnel B
```

De esta forma, dos desarrolladores o dos agentes de IA pueden trabajar simultáneamente sin compartir accidentalmente puertos, contenedores, dependencias o datos.

Los locks y la administración de recursos evitan además que operaciones incompatibles sobre el mismo target ocurran simultáneamente.

---

22. Transacciones y rollback

Las operaciones mutables utilizan un contexto transaccional basado en:

```
LifecycleTransaction
```

La finalidad es registrar las mutaciones realizadas y revertir de forma segura aquellas que sean compensables cuando ocurre un fallo.

El rollback no debe fingirse.

Si una mutación no puede revertirse con seguridad, el sistema debe conservar suficiente diagnóstico para indicar que existe un estado parcial o degradado.

---

23. Estado de madurez

El lifecycle está implementado en:

```
v0.1.0
```

Incluye:

```
init
adopt
destroy
list
status
doctor
start
stop
```

También dispone de:

```
capabilities
integrations
transactions
rollback
locks
path boundaries
suite de conformidad end-to-end
```

Las pruebas se encuentran principalmente en:

```
tests/unit/
tests/conformance/
```

El trabajo posterior registrado en `[Unreleased]` incorpora además:

```
schema versioning
workspace provisioning
after_init
after_adopt
runtime provisioning
bootstrap con Composer
data provisioning con PostgreSQL
paridad SiVale
```

---

24. Documento autoritativo

El documento de diseño autoritativo del proyecto es:

```
docs/standard-operativo.md
```

Ante dudas respecto a arquitectura, responsabilidades o comportamiento esperado, este documento debe considerarse la referencia principal.

---

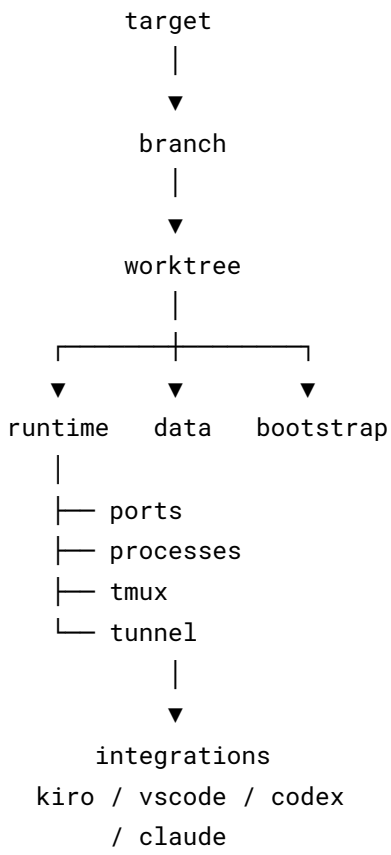
25. Resumen del flujo

La adopción de un proyecto puede verse así:

```
Instalar imass-wt
       ↓
Crear .imass-worktree.toml
       ↓
Configurar target / branch / worktree
       ↓
Habilitar capabilities necesarias
       ↓
imass-wt doctor
       ↓
imass-wt init <target>
       ↓
Trabajar en el workspace
```

```
       ↓
imass-wt status / list
       ↓
imass-wt stop / start
       ↓
imass-wt destroy <target>
```

Y el objetivo final es disponer de una forma única, reproducible y segura de administrar **ecosistemas de desarrollo temporales e independientes**:



`imass-worktree` proporciona así un lifecycle común:

```
init
adopt
status
list
start
stop
destroy
doctor
```

sobre el cual cada proyecto puede agregar únicamente las capacidades operativas que realmente necesita, manteniendo aislados código, runtime, puertos, dependencias, datos y exposición externa entre diferentes workspaces.